

Model-Assisted Widget Synthesis: Bridging Declarative UX and Runtime Adaptation

Table of Contents

1. Abstract
2. Introduction
3. Context and Problem Statement
4. Architecture Proposal
5. Core Components
6. Research Agenda
7. Prototype Scenarios
8. Evaluation and Validation
9. Conclusions

Abstract

This paper explores how model-assisted workflows can augment declarative UX by proposing widget scaffolding, domain-aware prompts, and structured adaptations without undermining compile-time validation. The key contribution is not raw generation, but grounded synthesis driven by typed tools, deterministic retrieval, and explicit state models.

This positions model assistance inside a live host rather than outside it. Suggestions can be generated, inspected, replayed, validated, and steered in session, which gives the paper a stronger and more distinctive contribution than a generic AI-for-UI framing.

Introduction

This chapter introduces model-assisted widget synthesis as a practical enhancement to declarative applications built on the platform. The goal is to use tools and knowledge resources to improve authoring and adaptation without compromising compile-time validation.

The main challenge is maintaining a distinction between interpolation and structured reasoning. Interpolation generates UI fragments by pattern matching over examples, while structured reasoning uses explicit metadata, tool outputs, and knowledge retrieval to compose behavior with known semantics. The platform's compile-first architecture is well suited to the latter.

This chapter also frames model-assisted synthesis in the context of platform plugins. `plugin-onnx` provides a deterministic knowledge index for domain content and prompt templates, while `plugin-agentic` contributes explicit planning and execution primitives. These plugins anchor the model-assisted workflow in the platform's existing extension model.

Finally, the chapter states that this paper is not about replacing authored UI with black-box generation. It is about using model-assisted tools to propose and validate changes in a disciplined, compile-first pipeline.

Context and Problem Statement

This chapter defines the specific problems that model-assisted widget synthesis addresses.

Static widget definitions often struggle to keep pace with evolving interaction patterns, domain-specific content, and adaptive presentation requirements. As applications grow, maintaining consistent behavior across forms, flows, and plugin-driven screens becomes harder.

The opportunity is to use structured metadata as the medium for assistance. Widget definitions on the platform already contain explicit state machines, form schemas, and plugin metadata. This structured input can be used to guide synthesis in a way that preserves validation and semantic fidelity.

The chapter identifies two concrete questions. First, how can the platform use knowledge-indexed search to select prompt templates and domain-specific behavior predictably? Second, how can the platform expose planning and intent metadata so that model-assisted workflows remain auditable and consistent with compiled semantics?

These questions are grounded in actual platform capabilities, not in generative speculation. They set the stage for an architecture that respects the platform's existing compile-first guarantees.

Architecture Proposal

This chapter describes a concrete architecture for model-assisted widget synthesis.

Declarative widget metadata is treated as model input. Source definitions include state declarations, transition guards, service invocation contracts, and plugin metadata. A model-assisted system can consume this structured input to generate or refine widget proposals.

Runtime adaptation is implemented through deterministic lookups and controlled suggestion layers. A build-time search index can provide compact, reproducible access to domain content and prompt templates, while linked-data or graph metadata can provide intent and entity relationships. These resources are accessed through explicit tools rather than hidden retrieval paths.

Fidelity constraints are enforced by the compiler. Proposed changes must pass the same schema validation, i18n resolution, and security checks as authored source. The model-assisted layer therefore functions as an advisor rather than a runtime executor.

The overlooked architectural insight is that suggestion quality improves when the host exposes live context: recent tool invocations, active session mode, current route, FSM snapshots, and plan state. That turns model assistance into a situated, auditable interaction rather than a detached code-generation pass.

Core Components

This chapter specifies the core components of a model-assisted pipeline.

A synthesis engine proposes new widget patterns based on existing metadata. It generates scaffolded definitions for forms, validation flows, and stateful widgets that conform to declared schemas and available plugin abstractions.

Local and remote inference are combined with validation. Local tools such as deterministic search indices and semantic graph lookups offer predictable retrieval. Remote model suggestions may provide broader context, but they are subject to the compiler's validation rules before acceptance.

A session layer is equally important. Agent sessions can run in blocking, queueing, or steering modes; plan state can be created and stepped explicitly; and invocation history can be retained as machine-readable evidence. Together with tools, prompts, and knowledge resources, these components form a typed cognitive surface that makes model assistance collaborative and revisable rather than one-shot.

Semantic diffing is essential. The compiler compares proposed changes with existing widget definitions at the level of states, transitions, and contracts. This prevents opaque alterations and ensures that developer review can focus on semantic differences.

Research Agenda

This chapter defines a research agenda for model-assisted widget synthesis.

Evaluating quality and consistency requires metrics for suggestion precision, compilation pass rate, author acceptance, and recovery after incorrect proposals. A useful study compares model-assisted proposals with manual authoring to determine whether assistance improves speed without sacrificing correctness.

A central research problem is distinguishing tool-grounded reasoning from interpolation. The agenda should separate proposals that rely on explicit metadata, live host state, search results, and plan context from proposals that merely imitate prior examples. One important metric is semantic fidelity to the declared schema and current session context.

Human-in-the-loop workflows are also a research priority. The paper should explore interfaces for presenting suggestions, capturing feedback, recording decisions, and replaying proposal histories so that future agents can learn from accepted and rejected changes.

Responsible guardrails remain necessary. The compiler should reject suggestions that violate security, accessibility, or policy constraints, while the host should preserve enough evidence to explain why a proposal was made and why it was accepted or rejected.

Prototype Scenarios

This chapter describes prototype scenarios that demonstrate model-assisted widget synthesis.

A prototype for form generation uses the platform form schema metadata to scaffold a validated widget, complete with fields, validation rules, and state transitions. The generated artifact is then compiled and verified against the toolchain.

A second prototype infers state machines from user stories and intent descriptions. The proposal maps intent into explicit states and transitions that match the platform's FSM model, rather than generating arbitrary markup.

A third prototype uses knowledge-aware styling and localization suggestions. `plugin-onnx` can provide domain-specific prompt titles or labels, while `plugin-rdf` can provide semantic tags for accessibility and content provenance.

A combined prototype integrates `plugin-onnx`, `plugin-agentic`, and `plugin-hosted` metadata into a workflow where knowledge and planning services are first-class inputs and compile-time validation remains the final arbiter.

Evaluation and Validation

This chapter defines evaluation and validation criteria for model-assisted synthesis.

Acceptance and trust are measured by review speed, modification rates, provenance quality, and developer confidence in suggestions. This reflects whether assistance is genuinely useful rather than merely novel.

The evaluation should also compare local deterministic synthesis with cloud-assisted suggestions and measure how well each approach exploits live host context such as session mode, plan state, recent tool outputs, and FSM snapshots.

Case studies should evaluate complete workflows in running applications. The selected cases should document compile-time verification, the fidelity of generated artifacts, the clarity of replayable reasoning traces, and how human steering changed the final outcome.

Conclusions

This chapter summarizes the practical conclusions of model-assisted widget synthesis.

The key finding is that model assistance is most credible when it is grounded in explicit metadata, typed tool contracts, deterministic retrieval, and a live host with observable state. In that setting, suggestions can be proposed, validated, replayed, and improved without replacing authored semantics.

The practical roadmap therefore includes deeper integration of prompt selection, semantic metadata, plan-state presentation, replayable proposal traces, and agent session controls. The result is not just smarter generation, but a more accountable environment for collaborative system evolution.